

Luniero Marketing Agent — Slash Command Interface

Design Philosophy

Users shouldn't learn a new interface. They already know: - /command syntax from Slack, Discord, Telegram - Natural language descriptions - Conversational follow-ups

Principle: /<action> <natural language description>

Command Categories

1. Content Creation

/write <what you want>

Examples:

```
/write a LinkedIn post about AI trends for 2026
/write a Twitter thread on why startups should use AI automation
/write a blog post about content marketing best practices
/write Instagram captions for our new product launch (5 variations)
/write email sequence for new client onboarding (3 emails)
/write ad copy for Facebook campaign targeting small business owners
```

What happens: 1. Agent parses intent (platform, content type, topic) 2. Loads client context (if client specified or default) 3. Runs: Brief → Draft → Polish → Review pipeline 4. Returns content + asks for approval

Flags:

/write ... --client acme	# Specify client
/write ... --platform linkedin	# Force platform
/write ... --tone casual	# Override tone
/write ... --length short	# Adjust length
/write ... --variations 3	# Multiple versions

2. Quick Content (Skip Full Pipeline)

/quick <what you want>

Examples:

```
/quick LinkedIn post about our new feature
/quick tweet celebrating 1000 followers
/quick Instagram story caption for team photo
```

What happens: - Skips full pipeline - Single LLM call with client context - Faster but less refined - Good for simple, time-sensitive content

3. Content Calendar

/calendar <timeframe> [platform]

Examples:

/calendar this week
/calendar next week linkedin
/calendar february instagram
/calendar q1 all

What happens: 1. Loads client content pillars 2. Researches trending topics 3. Generates content calendar 4. Returns schedule with post ideas

Output:

□ Week of Feb 10-14 – Acme Corp LinkedIn

Mon 10: Thought leadership – "Why async beats meetings"
Tue 11: Product tip – Hidden feature spotlight
Wed 12: Industry news – React to AI announcement
Thu 13: Customer story – Quote from happy client
Fri 14: Weekend thought – Reflection post

Approve this calendar? (yes/edit/regenerate)

4. Reports

/report <type> [timeframe]

Examples:

/report weekly
/report monthly
/report campaign "Summer Launch"
/report performance last 30 days
/report competitor analysis

What happens: 1. Pulls analytics from connected platforms 2. Compiles metrics 3. Generates insights and recommendations 4. Outputs formatted report (text + PDF option)

Output:

□ Acme Corp – Weekly Report (Feb 3-9)

HIGHLIGHTS

□ LinkedIn engagement up 23%
□ Best post: "AI trends" (2.4K impressions)
□ Twitter reach down 8%

TOP CONTENT

1. "Why we ditched spreadsheets" – 847 engagements
2. Product demo video – 412 views
3. Team spotlight – 234 likes

RECOMMENDATIONS

→ Double down on LinkedIn carousels
→ Test Twitter threads instead of links
→ Post customer stories on Wednesdays

Want the full PDF? (/report pdf)

5. Research

/research <topic or request>

Examples:

```
/research trending topics in B2B SaaS
/research what competitors are posting this week
/research best hashtags for project management content
/research viral posts in our industry
/research audience sentiment about AI tools
```

What happens: 1. Research Agent activates 2. Searches web, social platforms, news 3. Compiles findings 4. Returns actionable insights

6. Client Management

/client <action> [details]

Examples:

/client list	# Show all clients
/client switch acme	# Change active client
/client new "Initech Corp" B2B SaaS	# Create new client
/client voice	# Show current client's
brand voice	
/client voice update "more casual"	# Update brand voice
/client pillars	# Show content pillars
/client pillars add "sustainability"	# Add content pillar

Active Client: - One client is "active" at a time - All commands default to active client - Switch with /client switch <name> - Or specify per-command with --client

7. Approvals & Revisions

/approve [job_id]	# Approve pending content
/revise <feedback>	# Request changes
/reject [reason]	# Reject and explain why

Examples:

/approve	# Approve last pending item
/approve job_abc123	# Approve specific job
/revise make it more casual and add a statistic	
/revise stronger hook, remove the question at the end	
/reject off-brand, try again with different angle	

What happens: - /approve → Content moves to next stage or completion - /revise → Sends feedback to Draft/Polish agent, loops back - /reject → Logs feedback, starts fresh

8. Schedule & Publish

/schedule <when>	# Schedule approved content
/publish now	# Publish immediately
/queue	# Show scheduled posts

Examples:

/schedule tomorrow 9am	
/schedule monday 10am EST	
/schedule optimal	# Let AI pick best time
/publish now	
/queue	# See what's scheduled
/queue cancel job_abc123	# Cancel scheduled post

9. History & Status

/status [job_id]	# Check job status
/history [count]	# Recent jobs
/show <job_id>	# Show full output

Examples:

/status	# Status of last job
/status job_abc123	# Specific job
/history	# Last 10 jobs
/history 20	# Last 20 jobs
/show job_abc123	# Full content of a job

10. Help & Settings

/help [command]	# Get help
/settings	# Show current settings
/settings <key> <value>	# Update setting

Examples:

/help	# All commands
/help write	# Help for /write
/settings	# Show all settings
/settings default_platform linkedin	
/settings auto_approve false	
/settings timezone America/Chicago	

Conversational Mode

After any command, users can follow up naturally:

User: /write LinkedIn post about AI trends

Agent: Here's your post:
[content]

Approve? (yes/edit/regenerate)

User: make it shorter and punchier

Agent: Here's the revised version:

[shorter content]

User: perfect, schedule it for monday

Agent: Scheduled for Monday Feb 10, 9:00 AM. 📅

The agent understands context and follow-ups.

Command Aliases

Multiple ways to say the same thing:

Primary	Aliases
/write	/create, /draft, /make
/quick	/q, /fast
/calendar	/cal, /plan
/report	/analytics, /stats
/research	/find, /search
/client	/c
/approve	/ok, /yes, /lgtm
/revise	/edit, /change, /fix
/schedule	/sched, /post
/status	/s, /?
/help	/h, /?

Smart Parsing

The agent intelligently parses natural language:

/write linkedin post about our new feature

Parsed as:

```
{
  "command": "write",
  "platform": "linkedin",
  "content_type": "post",
  "topic": "our new feature",
  "client": "[active_client]"
}
```

/write 5 tweet variations about the product launch with casual tone

Parsed as:

```
{
  "command": "write",
  "platform": "twitter",
  "content_type": "tweet",
  "topic": "product launch",
  "variations": 5,
  "tone": "casual"
}
```

Interactive Prompts

When info is missing, agent asks:

User: /write a blog post

Agent: I'll create a blog post for you.

What topic should I cover?

User: AI automation for small businesses

Agent: Got it. Any specific angle or key points to include?

User: focus on cost savings and time efficiency

Agent: Creating your blog post now...
[runs pipeline]

Response Format

All responses follow a consistent format:

Content Output

□ **LinkedIn Post** – Acme Corp

[content here]

□ Words: 127 | Score: 94/100
□ #AIautomation #productivity

□ Approve ⇌ Revise □ Regenerate

Status Update

□ Job: job_abc123
□ Status: polishing
□ Client: Acme Corp
□ Type: LinkedIn post
□ Started: 2 min ago

Error

□ Error: Could not connect to LinkedIn API

Possible fixes:

1. Check API credentials in /settings
 2. Try again in a few minutes
 3. Use /help linkedin for setup guide
-

Implementation Architecture

Parser Module (src/cli/parser.ts)

```
interface ParsedCommand {
  command: string;
  subcommand?: string;
  topic?: string;
  platform?: string;
  contentType?: string;
  client?: string;
  flags: Record<string, any>;
  raw: string;
}

function parseCommand(input: string): ParsedCommand {
  // Handle slash commands
  if (input.startsWith('/')) {
    return parseSlashCommand(input);
  }

  // Handle conversational follow-ups
  return parseConversational(input);
}

function parseSlashCommand(input: string): ParsedCommand {
  const [command, ...rest] = input.slice(1).split(' ');
  const content = rest.join(' ');

  // Extract flags (--key value)
  const flags = extractFlags(content);
  const cleanContent = removeFlags(content);

  // Smart parsing based on command
  switch (command) {
    case 'write':
    case 'create':
    case 'draft':
      return parseWriteCommand(cleanContent, flags);
    case 'calendar':
    case 'cal':
      return parseCalendarCommand(cleanContent, flags);
    // ... etc
  }
}

function parseWriteCommand(content: string, flags: Record<string, any>): ParsedCommand {
  // Use NLP or pattern matching to extract:
  // - Platform (linkedin, twitter, instagram, etc.)
  // - Content type (post, thread, carousel, story, etc.)
  // - Topic
  // - Modifiers (casual, short, variations, etc.)

  const platform = extractPlatform(content) || flags.platform;
  const contentType = extractContentType(content) || 'post';
  const topic = extractTopic(content);

  return {

```

```

        command: 'write',
        platform,
        contentType,
        topic,
        flags,
        raw: content,
    };
}

```

Command Router (src/cli/router.ts)

```

    async function routeCommand(parsed: ParsedCommand, context:
SessionContext) {
        switch (parsed.command) {
            case 'write':
                return await handleWriteCommand(parsed, context);

            case 'quick':
                return await handleQuickCommand(parsed, context);

            case 'calendar':
                return await handleCalendarCommand(parsed, context);

            case 'report':
                return await handleReportCommand(parsed, context);

            case 'research':
                return await handleResearchCommand(parsed, context);

            case 'client':
                return await handleClientCommand(parsed, context);

            case 'approve':
                return await handleApproveCommand(parsed, context);

            case 'revise':
                return await handleReviseCommand(parsed, context);

            case 'schedule':
                return await handleScheduleCommand(parsed, context);

            case 'status':
                return await handleStatusCommand(parsed, context);

            case 'help':
                return await handleHelpCommand(parsed, context);

            default:
                // Try to understand as conversational
                return await handleConversational(parsed, context);
        }
    }
}

```

Session Context (src/cli/session.ts)

```

interface SessionContext {
    activeClient: string | null;
    lastJobId: string | null;
    pendingApproval: string | null;
}

```



```

    conversationHistory: Message[];
    settings: UserSettings;
}

class Session {
    private context: SessionContext;

    async processInput(input: string): Promise<Response> {
        // Parse the input
        const parsed = parseCommand(input);

        // Check for conversational context
        if (this.isFollowUp(parsed)) {
            return await this.handleFollowUp(parsed);
        }

        // Route to appropriate handler
        return await routeCommand(parsed, this.context);
    }

    private isFollowUp(parsed: ParsedCommand): boolean {
        // No slash command = likely follow-up
        if (!parsed.raw.startsWith('/')) {
            return this.context.conversationHistory.length > 0;
        }
        return false;
    }

    private async handleFollowUp(parsed: ParsedCommand):
    Promise<Response> {
        // Use LLM to understand intent in context
        const intent = await this.classifyIntent(parsed.raw);

        switch (intent) {
            case 'approve':
                return await handleApproveCommand({ command: 'approve' },
this.context);
            case 'revise':
                return await handleReviseCommand({
                    command: 'revise',
                    topic: parsed.raw
                }, this.context);
            case 'schedule':
                return await handleScheduleCommand({
                    command: 'schedule',
                    topic: parsed.raw
                }, this.context);
            default:
                return await handleConversational(parsed, this.context);
        }
    }
}

```

Write Command Handler (src/cli/handlers/write.ts)

```

async function handleWriteCommand(
    parsed: ParsedCommand,
    context: SessionContext
): Promise<Response> {
    // Get client

```

```

const clientId = parsed.flags.client || context.activeClient;
if (!clientId) {
  return {
    type: 'prompt',
    message: 'Which client is this for?',
    options: await listClients(),
  };
}

// Create job
const jobId = await routerAgent.createJob({
  clientId,
  type: mapContentType(parsed.contentType),
  platform: parsed.platform,
  topic: parsed.topic,
  instructions: parsed.flags.instructions,
});

// Update session
context.lastJobId = jobId;
context.pendingApproval = jobId;

// Wait for completion (with streaming updates)
return await streamJobProgress(jobId);
}

async function streamJobProgress(jobId: string): Promise<Response> {
  return new Promise((resolve) => {
    const checkInterval = setInterval(async () => {
      const job = await stateStore.getJob(jobId);

      // Send progress update
      emitProgress({
        jobId,
        status: job.status,
        stage: getStageLabel(job.status),
      });

      // Check if complete
      if (job.status === 'complete' || job.status ===
'human_review') {
        clearInterval(checkInterval);
        resolve(formatContentResponse(job));
      }

      if (job.status === 'failed') {
        clearInterval(checkInterval);
        resolve(formatErrorResponse(job));
      }
    }, 1000);
  });
}

```

Complete Command Reference

Content Commands

Command	Description	Example
/write <description>	Create content with full pipeline	/write LinkedIn post about AI
/quick <description>	Fast single-shot content	/quick tweet about our sale
/calendar <timeframe>	Generate content calendar	/calendar next week
/repurpose <source> to <target>	Convert content	/repurpose blog to twitter thread

Analytics Commands

Command	Description	Example
/report <type>	Generate report	/report monthly
/research <topic>	Research a topic	/research competitor content
/trending	Get trending topics	/trending in B2B SaaS

Client Commands

Command	Description	Example
/client list	List all clients	/client list
/client switch <name>	Change active client	/client switch acme
/client new <name> <industry>	Create client	/client new "Acme" SaaS
/client voice	View brand voice	/client voice
/client pillars	View content pillars	/client pillars

Workflow Commands

Command	Description	Example
/approve	Approve pending content	/approve
/revise <feedback>	Request changes	/revise make it shorter
/reject <reason>	Reject content	/reject off-brand
/schedule <when>	Schedule content	/schedule monday 9am
/publish	Publish immediately	/publish now

Utility Commands

Command	Description	Example
/status	Check job status	/status
/history	View recent jobs	/history 10
/queue	View scheduled posts	/queue
/help	Get help	/help write
/settings	View/update settings	/settings timezone EST

Error Handling

Missing Information

User: /write a post

Agent: I'd love to help! A few quick questions:

1. What platform? (LinkedIn, Twitter, Instagram, etc.)
2. What topic should the post cover?

Or just tell me naturally: "LinkedIn post about our new feature"

Invalid Command

User: /postify trending content

Agent: I don't recognize `/postify`. Did you mean:

- `/write` – Create content
- `/quick` – Quick content
- `/research` – Research topics

Type `/help` for all commands.

Processing Error

User: /report monthly

Agent: ☐ Couldn't generate the report.

Issue: LinkedIn API returned an error (rate limited)

Options:

1. Try again in 15 minutes
 2. Generate report without LinkedIn data: `/report monthly --skip linkedin`
 3. Check API status: `/status apis`
-

File Structure for CLI Module

```
src/cli/
├── index.ts           # Main entry point
├── parser.ts          # Command parsing
├── router.ts          # Command routing
├── session.ts         # Session management
├── formatter.ts       # Response formatting
├── handlers/
│   ├── write.ts       # /write command
│   ├── quick.ts       # /quick command
│   ├── calendar.ts    # /calendar command
│   ├── report.ts      # /report command
│   ├── research.ts    # /research command
│   ├── client.ts      # /client commands
│   ├── approval.ts    # /approve, /revise, /reject
│   └── schedule.ts    # /schedule, /publish, /queue
```

```
|   |—— status.ts      # /status, /history
|   |—— help.ts        # /help, /settings
|—— utils/
|   |—— nlp.ts          # Natural language parsing
|   |—— streaming.ts    # Progress streaming
|   |—— prompts.ts      # Interactive prompts
```

Copy this entire document into Claude Code and say: “Add this CLI interface to the Luniero implementation. Update the project structure and implement the parser, router, and handlers.”